

# การออกแบบและวิเคราะห์อัลกอริทึมด้วย STACK

การนำรถเป้าหมายออกจากลานจอดรถที่มีทางเข้าออกเพียงด้านเดียว

# สมาชิก

- นายสรสิทธิ์ มาตรฐาน 68122250006
- นาย เอกราช บุตรรักษ์ 68122250007
- นาย ภาพตะวัน สายแก้ว 68122250012
- นางสาว ณัฐชา ขำทองทับ 68122250037

# Case Story

- ลานจอดรถแห่งนี้มีทางเข้าและทางออกเพียงด้านเดียว ทำให้รถที่เข้ามาก่อนจะอยู่ลึกเข้าไปด้านใน ส่วนรถที่เข้ามาทีหลังจะอยู่ใกล้ทางออกมากกว่า
- เมื่อต้องการนำรถคันใดคันหนึ่งออกจากลานจอด รถที่ขวางอยู่ด้านหน้าจะต้องถูกนำออกชั่วคราวก่อน เมื่อพบรถเป้าหมายและนำออกแล้ว รถที่ถูกย้ายออกชั่วคราวจะต้องถูกนำกลับเข้ามายังลานจอดในลำดับเดิม เพื่อรักษาสภาพการจอดรถให้เหมือนก่อนการนำรถเป้าหมายออก

# Input, Output ,Constraints

- **Input**

- **License Plate**
- **Owner Name**
- **Vehicle Type**
- **Target Plate**

- **Output**

- **Found / Not Found**
- **Cars Moved**
- **Push**
- **Pop**
- **Execution Time**

- **Constraints**

- **ใช้ Stack**
- **รถห้ามหาย**
- **ต้องรักษาลำดับเดิม**
- **มี Algorithm 2 วิธี**

# Algorithm A

1. Create ParkingRecord
2. While parkingStack is not empty
  - 2.1 Pop a vehicle from parkingStack
  - 2.2 Increase Pop Count
  - 2.3 Increase Comparison Count
  - 2.4 If vehicle license plate equals target
    - Set found = true
    - Stop searching
  - Else
    - Push vehicle into temporaryStack
    - Increase Push Count
    - Increase Cars Moved
3. While temporaryStack is not empty
  - 3.1 Pop vehicle from temporaryStack
  - 3.2 Push vehicle back into parkingStack
  - 3.3 Increase Push Count
4. Return ParkingRecord

- ใช้งาน Stack จำนวน 2 ตัว
  - parkingStack
  - temporaryStack

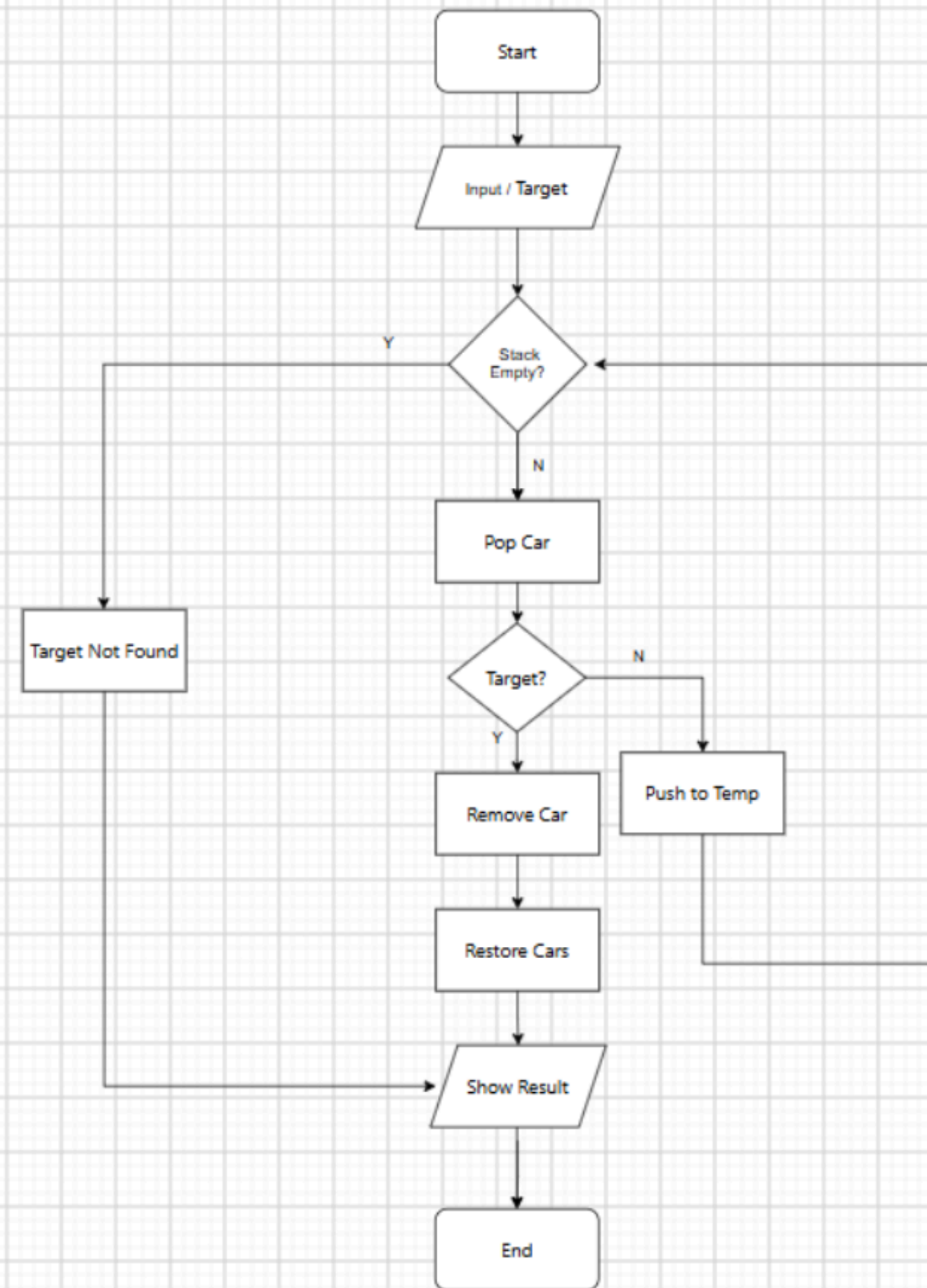


# Algorithm B : Recursive Method

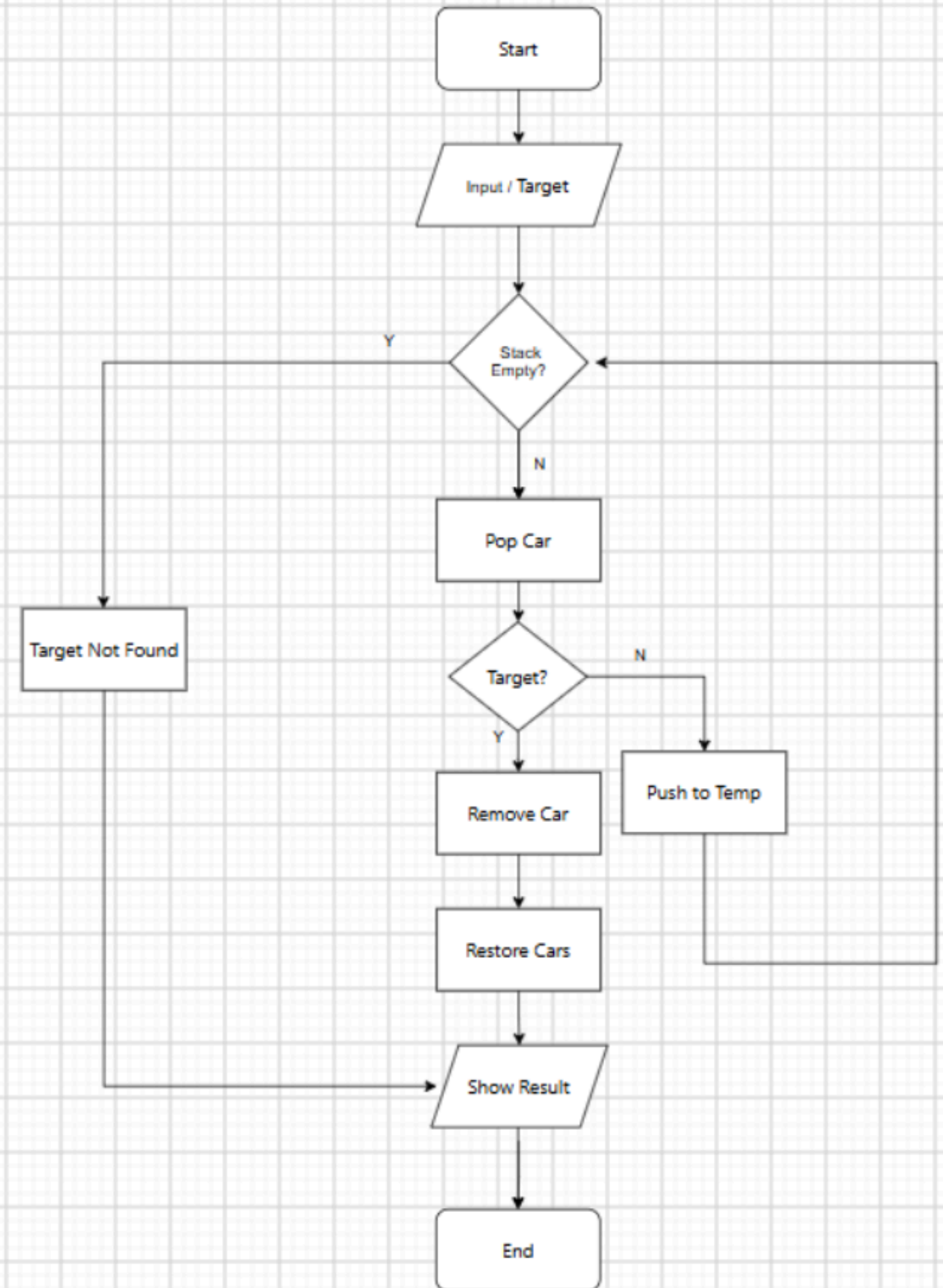
1. Create ParkingRecord
2. Call RecursiveSearch(target, ParkingRecord)  
RecursiveSearch(target, record)
3. If parkingStack is empty  
Return false
4. Increase Loop Count
5. Pop a vehicle from parkingStack
6. Increase Pop Count
7. Increase Comparison Count
8. If vehicle license plate equals target  
Set found = true  
Return true
9. Call RecursiveSearch(target, record)
10. Push the removed vehicle back into parkingStack
11. Increase Push Count
12. If target was not found  
Increase Cars Moved
13. Return search result

- Input: target license plate
- Output: ParkingRecord

# Flowchart Algorithm A



# Flowchart Algorithm B





# Loop Invariant

- ก่อนเริ่มการทำงานในแต่ละรอบของลูป  
รถทุกคันที่ถูกนำออกจาก parkingStack และไม่ใช่รถเป้าหมาย จะถูกเก็บไว้ใน temporaryStack โดยไม่มีข้อมูลสูญหาย และรถที่ยังอยู่ใน parkingStack ยังคงลำดับเดิม

- Initialization  
ก่อนเริ่มลูป  
temporaryStack ว่าง  
ดังนั้น Invariant เป็นจริง

- Maintenance  
เมื่อ Pop รถออกจาก parkingStack  
หากไม่ใช่รถเป้าหมาย  
temporaryStack.push(car)  
รถถูกเก็บครบถ้วน ไม่มีข้อมูลสูญหาย  
• ดังนั้น Invariant ยังคงเป็นจริง

Termination  
ลูปจะจบเมื่อ  
พบรถเป้าหมาย

- หรือ  
Stack ว่าง  
หลังจากนั้นนำรถทั้งหมดใน temporaryStack กลับไปยัง  
parkingStack  
ลำดับเดิมจึงถูกรักษาไว้

# สรุปผลการศึกษา

- โครงการนี้ได้ศึกษาและพัฒนาอัลกอริทึมการนำรถเข้าหมายออกจากลานจอดรถโดยใช้โครงสร้างข้อมูลแบบ Stack
- ได้ออกแบบอัลกอริทึม 2 วิธี ได้แก่
- Algorithm A : Iterative Method
- Algorithm B : Recursive Method
- ทั้งสองวิธีสามารถค้นหาและนำรถเข้าหมายออกได้ถูกต้อง พร้อมรักษาลำดับของรถที่เหลือให้คงเดิม
- ผลการวิเคราะห์พบว่าอัลกอริทึมทั้งสองมี Time Complexity เท่ากัน คือ  $O(n)$  และ Space Complexity เท่ากับ  $O(n)$